Fundamentals of Data Structures

Prof. Pranjal Pandit

Unit 1 Introduction to Algorithms and Data Structures

- From Problem to Program (Problem, Solution, Algorithm, Data Structure and Program).
- Data, Information, Knowledge and Data Structure;
- Abstract Data Types (ADT);
- Classification (Linear/Non-linear, Static/Dynamic, Persistent/Ephemeral).
- Algorithms: problem solving, characteristics, design tools (pseudo-code, flowchart).
- Complexity: space and time complexity, asymptotic notation (Big-O, Theta, Omega),
- Step-count method, analysis of Linear, Quadratic, Cubic and Logarithmic constructs.
- Algorithmic strategies: Divide and Conquer, Greedy.

Asymptotic Notations

- Asymptotic notations **are mathematical tools** used in computer science **to describe the behavior of algorithms as the input size grows.**

- They provide a way to express the efficiency of an algorithm in terms of time complexity and space complexity, focusing on the dominant factors that affect performance for large input sizes.

How to find the Time Complexity or running time for performing the operations?

- The measuring of the actual running time is not practical at all.
- **The running time to perform any operation depends on the size of the input**.

- Suppose we have an array of five elements, and we want to add a new element at the beginning of the array. To achieve this, we need to shift each element towards right, and suppose each element takes one unit of time. There are five elements, so five units of time would be taken.
- Suppose there are 1000 elements in an array**, then it takes 1000 units of time** to shift.
- **It concludes that time complexity depends upon the input size**
- Therefore, if the input size is n, then f(n) is a function of n that denotes the time complexity.

How to calculate f(n)?

- Calculating the value of f(n) **for smaller programs is easy but for bigger programs, it's not that easy.**
- We can compare the data structures by comparing their f(n) values.

Example.

f(n) = 5n2 + 6n + 12

where n is the number of instructions executed, and it depends on the size of the input

When n=1

- % of running time due to 5n2 = $\frac{5}{5+6+12}$ * 100 = 21.74%
- % of running time due to 6n = $\frac{6}{5+6+12}$ * 100 = 26.09%
- % of running time due to 12 = $\frac{12}{5+6+12}$ * 100 = 52.17%

Let's assume the different values of n to find the growth rate of f(n).

n	$5n^2$	6n	12
1	21.74%	26.09%	52.17%
10	87.41%	10.49%	2.09%
100	98.79%	1.19%	0.02%
1000	99.88%	0.12%	0.0002%

- As we can observe in the above table that with the increase in the value of n, the running time of $5n^2$ increases while the running time of 6n and 12 decreases.
- Therefore, it is observed that for larger values of n, the squared term consumes almost 99% of the time.
- **As the n2 term is contributing most of the time**, so we can eliminate the rest two terms.

Therefore,

$$\mathbf{f(n) = 5n^2}$$

- *Here, we are getting the approximate time complexity whose result is very close to the actual result.*
- And ***this approximate measure of time complexity is known as an Asymptotic complexity.***
- Here, we are not calculating the exact running time, ***we are eliminating the unnecessary terms, and we are just considering the term which is taking most of the time.***

Using the asymptotic analysis, we can easily conclude the average-case, best-case and worst-case scenario of an algorithm.

It is used to mathematically calculate the running time of any operation inside an algorithm.

The commonly used asymptotic notations used for calculating the running time complexity of an algorithm is given below:

- Big oh Notation (O)
- Omega Notation (Ω)
- Theta Notation (θ)

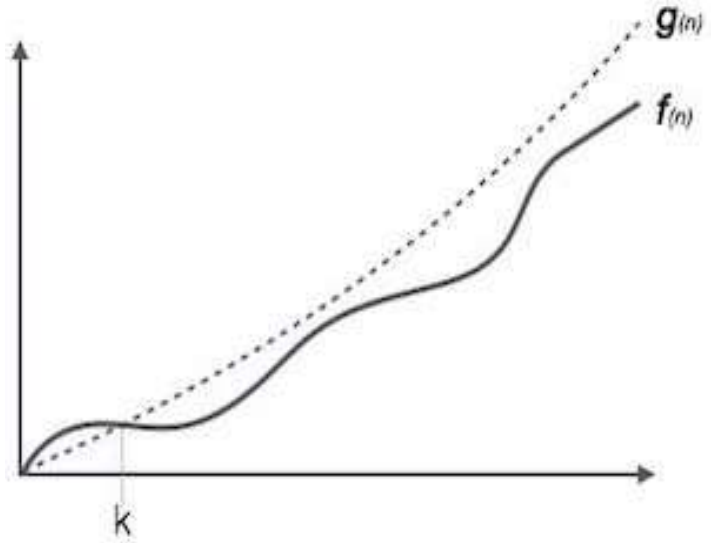
Big oh Notation (O)

- Big O notation is an asymptotic notation that measures the performance of an algorithm **by simply providing the order of growth of the function**.
- This notation **provides an upper bound on a function** which ensures that the **function never grows faster than the upper bound**. So, it gives the least upper bound on a function so that the function never grows faster than this upper bound.
- It is the formal way to express the upper boundary of an algorithm running time. It measures the worst case of time complexity or the **algorithm's longest amount of time to complete its operation**. It is represented as shown below:

For example:

- If **f(n)** and **g(n)** are the two functions defined for positive integers,
- then **f(n)** = **O(g(n))** as **f(n) is big oh of g(n)** or f(n) is on the order of g(n)) if there exists constants c and no such that:

f(n)≤c.g(n) for all n≥no



This ***implies that f(n) does not grow faster than g(n),*** or g(n) is an upper bound on the function f(n). In this case, we are calculating the growth rate of the function which eventually calculates the worst time complexity of a function, i.e., **how worst an algorithm can perform**

Let's understand through examples

Example 1: f(n)=2n+3 , g(n)=n

Now, we have to find **Is f(n)=O(g(n))?**

- To check f(n)=O(g(n)), it must satisfy the given condition:

f(n)<=c.g(n)

- First, we will replace f(n) by 2n+3 and g(n) by n.

2n+3 <= c.n

- **Let's assume c=5, n=1 then**

2*1+3<=5*1

5<=5

- **For n=1, the above condition is true.**

If n=2

2*2+3<=5*2
7<=10

For n=2, the above condition is true.

We know that for any value of n, it will satisfy the above condition, i.e., 2n+3<=c.n.

If the value of c is equal to 5, then it will satisfy the condition 2n+3<=c.n.

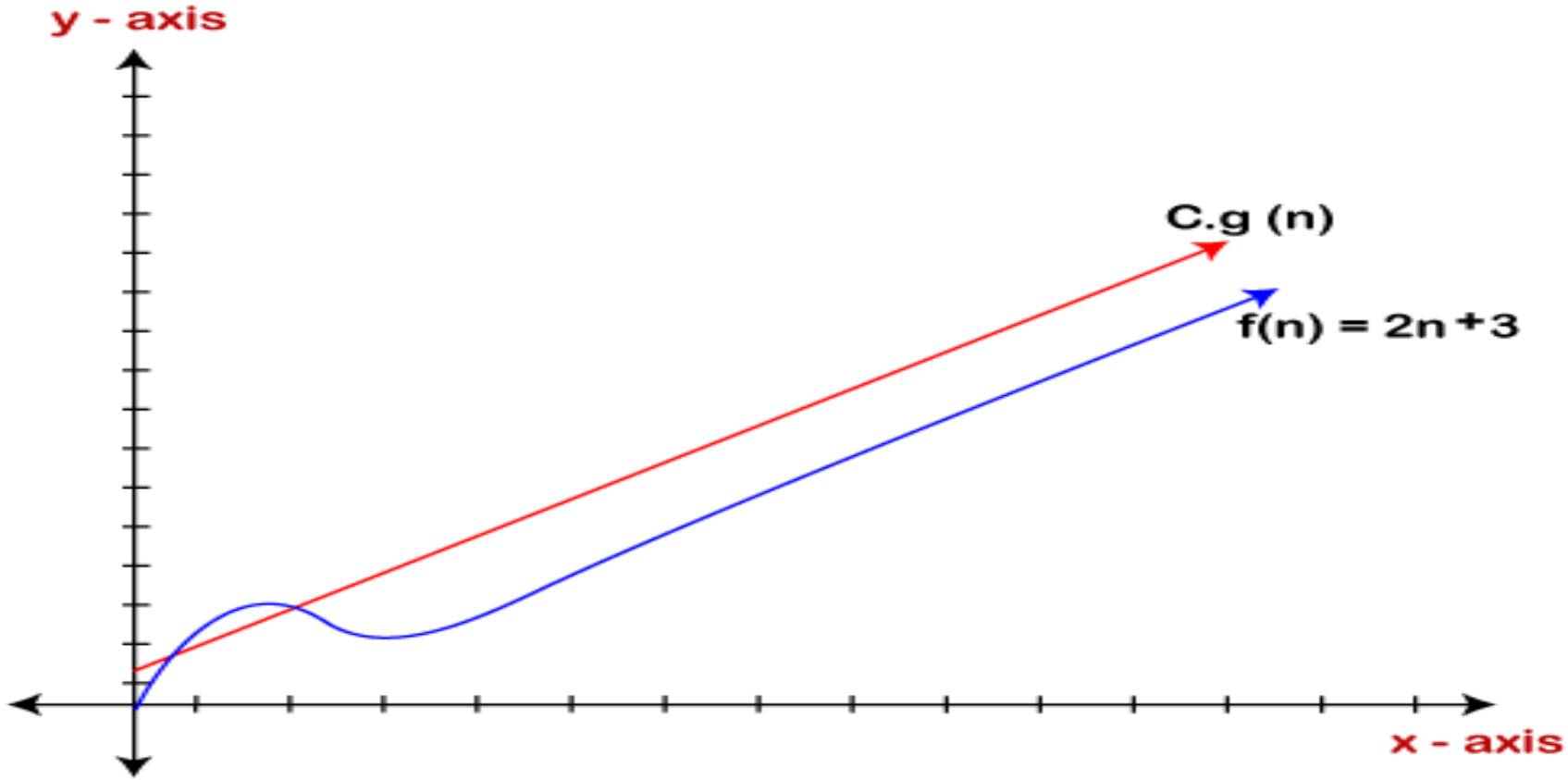
We can take any value of n starting from 1, it will always satisfy.

Therefore, we can say that for some constants c and for some constants n0, it will always satisfy 2n+3<=c.n.

As it is satisfying the above condition, so f(n) is big oh of g(n) or we can say that f(n) grows linearly.

Therefore, it concludes that c.g(n) is the upper bound of the f(n). It can be represented graphically as:

bound of the f(n). It can be represented graphically as:



The idea of using big o notation is to give an upper bound of a particular function, and eventually it leads to give a worst-time complexity

Omega Notation (Ω)

- It basically describes the **best-case scenario** which is opposite to the big o notation.
- It is the formal way to **represent the lower bound of an algorithm's running time.**
- It measures the best amount of time an algorithm can possibly take to complete or the best-case time complexity.
- It **determines what is the fastest time that an algorithm can run**.

If **f(n)** and **g(n)** are the two functions defined for positive integers,

then f(n) = Ω (g(n)) as **f(n) is Omega of g(n)** or f(n) is on the order of g(n)) if there exists constants c and no such that:

f(n)>=c.g(n) for all n≥no and c>0

Let's consider a simple example.

If f(n) = 2n+3, g(n) = n,

Is f(n)= **Ω** (g(n))?

It must satisfy the condition:

f(n)>=c.g(n)

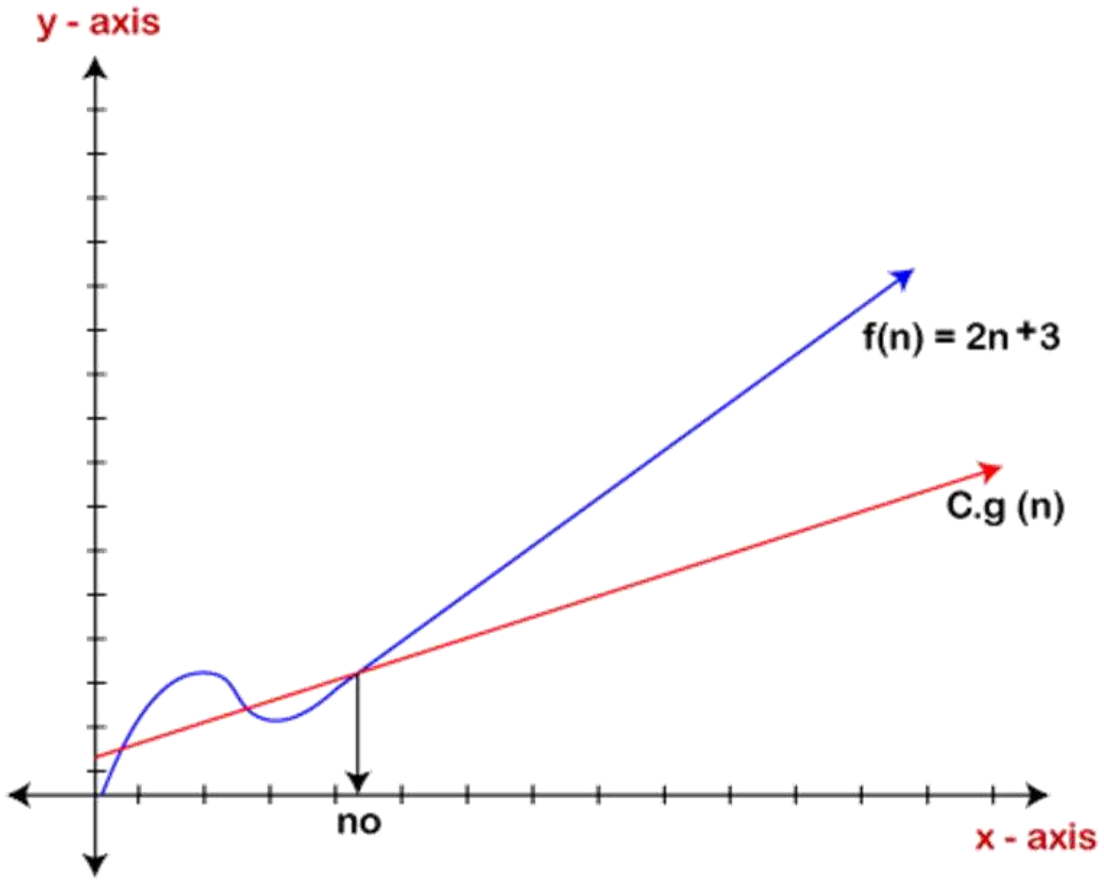
To check the above condition, we first replace f(n) by 2n+3 and g(n) by n.

2n+3>=c*n

Suppose c=1

2n+3>=n (This equation will be true for any value of n starting from 1).

Therefore, it is proved that g(n) is big omega of 2n+3 function.



Theta Notation (θ)

- The theta notation mainly describes the **average case scenarios**.
- It represents the **realistic time complexity** of an algorithm. Every time, an algorithm does not perform worst or best, in real-world problems, algorithms mainly fluctuate between the worst-case and best-case, and this gives us the average case of the algorithm.
- Big theta is mainly used when the value of worst-case and the best-case is same.
- It is the formal way to express both the upper bound and lower bound of an algorithm running time.

Let's understand the big theta notation mathematically:

Let f(n) and g(n) be the functions of n where n is the steps required to execute the program then:

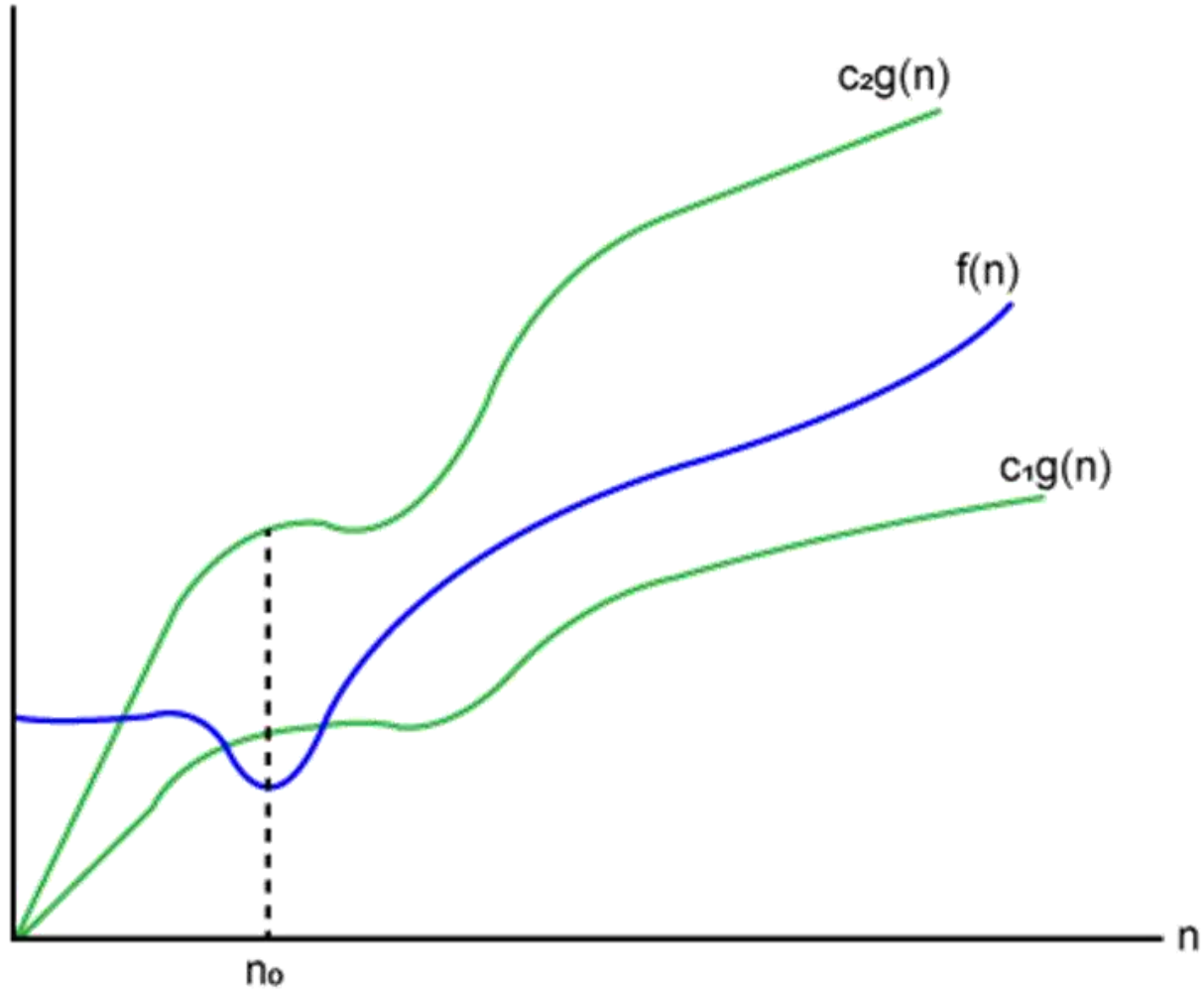
$$\mathbf{f(n)= \theta g(n)}$$

The above condition is satisfied **only if when**

$$\mathbf{c1.g(n)<=f(n)<=c2.g(n)}$$

where the **function is bounded by two limits**, i.e., upper and lower limit, and f(n) comes in between.

The condition **f(n)= θg(n)** will be true if and only if c1.g(n) is less than or equal to f(n) and c2.g(n) is greater than or equal to f(n).



$f(n) = \Theta(g(n))$

Let's consider the same example where

f(n)=2n+3

g(n)=n

As **c1.g(n) should be less than f(n) so c1 has to be 1** whereas **c2.g(n) should be greater than f(n) so c2 is equal to 5.**

The c1.g(n) is the lower limit of the of the f(n) while c2.g(n) is the upper limit of the f(n).

c1.g(n)<=f(n)<=c2.g(n)

Replace g(n) by n and f(n) by 2n+3

c1.n <=2n+3<=c2.n

if c1=1, c2=2, n=1

1*1 <=2*1+3 <=2*1

1 <= **5** <= **2** // **for n=1, it satisfies the condition** c1.g(n)<=f(n)<=c2.g(n)

If n=2

1*2<=2*2+3<=2*2

2<=7<=4 // **for n=2, it satisfies the condition** c1.g(n)<=f(n)<=c2.g(n)

Therefore, we can say that for any value of n, it satisfies the condition c1.g(n)<=f(n)<=c2.g(n).

Hence, it is proved that f(n) is big theta of g(n).

So, this is the average-case scenario which provides the realistic time complexity.

Why we have three different asymptotic analysis?

As we know that big omega is for the best case, big oh is for the worst case while big theta is for the average case. Now, we will find out the average, worst and the best case of the linear search algorithm.

- Suppose we have **an array of n numbers**, and we want to find the particular element in an array using the **linear search**.
- In the linear search, every element is compared with the searched element on each iteration.
- Suppose, **if the match is found in a first iteration only**, then the best case would be **Ω(1),** if the element matches with the last element, i.e., nth element of the array then the **worst case would be O(n**).
- The average case is the mid of the best and the worst-case, so it becomes **θ(n/1). The constant terms can be ignored in the time complexity so average case would be θ(n)**.

```
int binarySearch(int arr[], int l, int r, int x)
 {
    while (l <= r) {                    // 1. O(log n)
        int m = l + (r - l) / 2;        // 2. O(1)
        if (arr[m] == x)                // 3. O(1)
            return m;                   // 4. O(1)
        if (arr[m] < x)                 // 5. O(1)
            l = m + 1;                  // 6. O(1)
        else
            r = m - 1;                  // 7. O(1)
    }
    return -1;                          // 8. O(1)
}
```

```
int foo(int n) {
    int count = 0;                  // 1. O(1)
    for (int i = 1; i < n; i *= 2) // 2. O(log n)
    {
        count++;                    // 3. O(1)
    }
    return count;                   // 4. O(1)
}
```

Below are some examples that show how to determine f(n) for different types of code segments.

Example 1: Simple Loop

```
for (int i = 0; i < n; i++)
{
        // Some constant time operation
}
```

Analysis:

•The loop runs from i=0 to i=n−1, so it executes n times.

•The operation inside the loop takes constant time O(1).

f(n)=O(n)

Example 2: Nested Loops

```
for (int i = 0; i < n; i++)
 {
    for (int j = 0; j < n; j++)
    {
        // Some constant time operation
    }
}
```

Analysis:

- The outer loop runs n times.
- For each iteration of the outer loop, the inner loop also runs n times.
- The operation inside the inner loop takes constant time O(1).

f(n) = O(n^2)

Example 3: Logarithmic Loop

```
for (int i = 1; i < n; i *= 2)
{
    // Some constant time operation
}
```

Analysis:

- **-** The loop variable (i) starts at 1 and is **multiplied by 2 each time**.
- **-** The loop condition is (i < n), so the loop runs until (i) exceeds (n).
- **-** The number of iterations is approximately (log_2 n).

$$\mathbf{f(n) = O(\log n)}$$

How to identify Logarithmic time complexity?

- The term log n refers to the logarithm of n to a specific base.
- In computer science, unless otherwise specified, the logarithm is typically base 2 (log2n).

- This is because many logarithmic algorithms involve **dividing the problem into two parts**, such as in **binary search.**

- Mathematically, O(log n) indicates that the time complexity grows logarithmically with the size of the input n.

- In other words, **if the input size n doubles**, **the number of operations required only increases by a constant amount.**

- You can identify logarithmic time complexity in **algorithms that reduce the problem size by a constant factor (often by half) in each step**, rather than by a constant amount.

The logarithmic function **arises naturally when you repeatedly divide a problem into smaller parts (or it doubles)** especially when the size of the part you're working with decreases exponentially relative to the original size.

Thus, O(log n) complexity means that the number of operations needed to solve the problem grows logarithmically with the input size n.

This results in very efficient algorithms, especially for large inputs.

Example 4: Divide and Conquer (Merge Sort)

```
void mergeSort (int arr[], int l, int r)
{
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
```

Analysis:

- - The `mergeSort` function divides the array in half each time, so the depth of the recursion is (log n).
- - At each level of recursion, the `merge` function is called, which takes (O(n)) time.

$$\mathbf{f(n) = O(n \log n)}$$

Example 5: Linear Recursion

```
int fibonacci(int n) {
    if (n <= 1)                 // 1. O(1)
        return n;               // 2. O(1)
    return fibonacci(n - 1) +   // 3. O(2^n)
           fibonacci(n - 2);    // 4. O(2^n)
}
```

Analysis:

- **-** The `fibonacci` function **calls itself twice for each function call.**
- **-** This results in an exponential growth in the number of calls.

f(n) = O(2^n)

- The time complexities **O(n log n) and O(logn)** represent different growth rates of an algorithm's running time relative to the input size n.
- Understanding the difference between these two complexities is crucial in analyzing and comparing the efficiency of algorithms.

1. O(log n) - Logarithmic Complexity

- **Growth Rate:** Logarithmic time complexity grows very slowly as n increases. If you double the input size, the running time increases by only a small, constant amount.
- **Typical Algorithms:**
  - **Binary Search:** As discussed earlier, the search space is halved in each step, leading to O(log n) complexity.
  - **Tree Operations:** Searching, inserting, or deleting in a balanced binary search tree (BST) generally takes O(log n) time because the height of the tree is log n.

Example: If **n=1,000,000**, then **log2 n is about 20**. Even with large inputs, the running time remains manageable.

2. O(n log n) - Linearithmic Complexity

Growth Rate: Linearithmic time complexity is a combination of **linear and logarithmic** growth rates.

This means the running time grows faster than O(logn) but slower than O(n2).

- **Typical Algorithms:**
  - **Merge Sort:** An efficient sorting algorithm that divides the array in half, sorts each half, and then merges them.

    The **division leads to log n** factor, and the **merging leads to the n factor**, resulting in **O(n log n)** complexity.
  - **Heapsort:** Another sorting algorithm with similar O(n log n) complexity.

- **Example:** If **n=1,000,000, then n log2 n is about 20 million**. This is significantly larger than log n, meaning the algorithm will take longer to run.

Steps to Determine f(n):

1. Identify the **input size** (n) in the code.

2. **Analyze each loop or recursive call** to determine how many times it runs relative to (n).

3. **Sum up** the time complexities of different code sections **if they are executed sequentially**.

4. **Multiply** the time complexities if they are **nested** (like in nested loops).

5. **Ignore lower-order** terms and constant factors to find the asymptotic behavior.

- By analyzing the code in this way, you can determine f(n) and subsequently calculate the appropriate asymptotic notation.